

AC-Coach：面向“程序设计实习”课程的智能助教

课程大作业报告

组号：122 | 天师府分队 | 小组成员：蔡东辰 田特 刘家宏 | 2026 年 7 月 5 日

1 程序功能介绍

1.1 项目定位与背景

AC-Coach 是一款面向“程序设计实习”课程的 Qt 桌面端学习辅助工具。在程序设计实习中，学生真正的困难往往不是“不会写一行代码”，而是“不知道哪里没写对”“不知道为什么写不对”，以及在之后再次犯下同样的错误。市面上的通用 IDE 只解决“把代码写出来、跑起来”的问题，通用 AI 问答工具则容易直接给出完整答案，反而绕过了学生的思考过程。

因此，AC-Coach 的定位既不是通用 IDE，也不是代写作业工具，而是一个**面向初学者理解过程的智能助教**：它把学习者在这门课上的完整学习链条——读题、写代码、运行、调试、理解错误、积累错因、期末复习——整合到同一个桌面程序中，帮助后来者一站式地学好这门课。与三类常见工具相比，其差异如表 1 所示。

表 1: AC-Coach 与常见工具的定位差异	
对比对象	AC-Coach 的差异
普通 AI 问答工具	更贴近具体课程作业，注重引导而非直接给答案
普通 IDE	更关注初学者对错误的理解过程，而非工程功能
普通错题本	能保存真实的代码错误与完整调试过程，并且直接保存所有相关信息

本项目在功能设计上特意做了“减法”：不追求商业级 IDE 功能、大型工程管理与自动代写完整作业等，这是为了更好地服务于其核心目的。

1.2 总体功能概览

AC-Coach 的核心特色不在于“做一个编辑器”，而在于围绕课程学习建立一条完整的学习闭环：从读题录入开始，经编码、运行、调试、错误复盘，最终沉淀为错因库并服务于期末复习，而复习中生成的题目又可引导学生回到读题与编码，形成正向循环。

在实现上，系统对应地划分为两大工作模式：**面向日常写题的编码模式（Coding）**，以及面向错因沉淀与考前复习的**复习模式（Review）**，用户可在主界面一键切换。下面分别介绍。

1.3 编码模式功能

编码模式是一个围绕“单题练习”组织的轻量 IDE，主要提供以下能力：

- 题目管理与题面分析**：学生把课程题目原文粘贴录入后，可将其与当前 .cpp 文件绑定；系统在后台调用大模型把题面结构化（标题、背景、输入输出、样例、提示等），并自动分析题目摘要、涉及的知识点、常见坑点与难度，供随后查看。

- **代码编辑与文件管理**：提供带 C++ 语法高亮的多标签编辑器，支持打开项目文件夹、浏览文件树、新建/打开/保存文件，以及撤销、重做、剪切、复制、粘贴等常规编辑操作。
- **编译与运行**：一键调用本机 g++ 编译当前文件，编译日志输出到底部 Compile Log，并将 error/warning/note 解析进 Problems 表格、可点击定位；编译成功后在外部窗口运行可执行文件。
- **AI 助教分步调试（核心）**：当程序报错、输出不符或思路卡住时，学生可主动“召唤助教”。助教提供三种模式——自动判断、纠错调试、下一步提示，并把一次错误拆解为多个步骤，配合代码行高亮逐步引导学生理解错误来源，而非直接重写代码。三种模式的适用场景见表 2。

表 2: AI 助教的三种模式

模式	适用场景
自动判断	由模型结合代码、报错、输出与题目信息，自动决定进入纠错调试还是下一步提示
纠错调试	代码已基本写完，但出现编译错误、运行错误、输出不一致或 OJ 判错等情况
下一步提示	尚未写完或不知如何继续时，只给一个小提示，且要求学生先修改代码再继续

1.4 复习模式功能

复习模式把编码模式中积累的错误转化为长期可复用的学习资料，形成“越用越懂你”的个人错题本：

- **个人错因库**：每次助教诊断结束后，系统自动记录本次的错误类型并生成错因卡片，例如数组越界、循环边界错误、状态更新顺序错误、递归出口错误、输出格式错误等；卡片支持按知识点、掌握情况等条件筛选查看。
- **复习材料生成**：系统可根据历史错误，由 AI 把曾经写错的代码片段整理成结构化的错因总结与复习材料，帮助学生集中回顾薄弱环节。
- **考题生成与自动判题**：结合错因库，AI 可智能设计程序填空题与代码重写题，甚至把学生之前写错的代码重新“挖空”让其填写；作答的代码答案会在隐藏测试用例上自动编译判题，给出对错反馈，实现考前的针对性复盘。

1.5 典型使用案例：调试“魔兽世界”作业

以课程经典的“魔兽世界”类作业为例，这类题目通常涉及多个阵营、角色、武器、生命值、攻击规则、时间顺序与复杂的输出格式。很多同学并非完全写不出来，而是容易在对象关系、状态更新或输出顺序上出错。下面通过一次完整流程说明 AC-Coach 的价值：

小明把题目粘贴进 AC-Coach 后，系统识别出题目涉及的关键点；他开始编写代码，运行后发现输出与样例不一致，却分不清究竟是角色生成顺序错了、生命值更新错了，还是仅仅输出格式有问题。于是他点击“召唤助教”，系统结合题目、当前代码与运行结果，生成一次**分步骤**调试说明：

1. 系统高亮负责“武士”生成的方法，请小明复述这段代码的意图，并提示此处可能有问题；

- 小明未看出错误，点击“下一步”，系统自动跳转并高亮生命值扣减逻辑，指出按照前面武士生成的写法，此处扣减为何会出错——例如因为缺少虚函数，扣减时实际调用的是基类方法；
- 系统不替学生重写代码，而是点出错因，并把本次错误的相关信息与原因记录进错因库，供日后复习。

这样，学生经历的不再是简单的“AI 给答案”，而是一次接近助教现场讲解的调试复盘。

2 项目各模块与类设计细节

2.1 总体架构

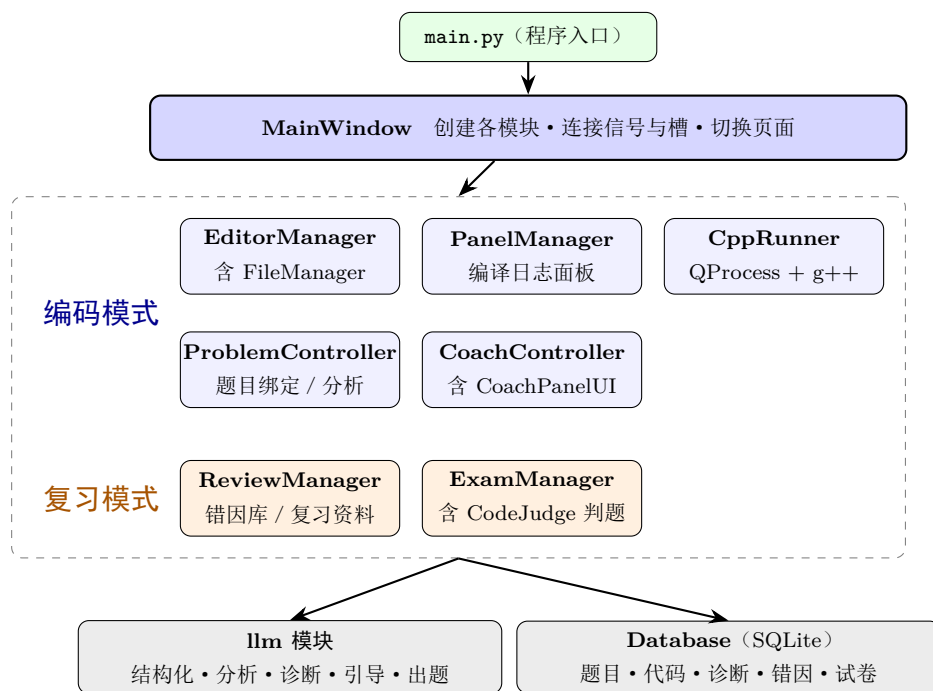


图 1: AC-Coach 总体架构框图（蓝色为编码模式模块，橙色为复习模式模块；箭头表示创建 / 调用与依赖方向）

2.2 主窗口与界面层

MainWindow 继承自 QMainWindow，是主窗口与各功能模块的组织者：初始化时经 `Ui_MainWindow.setupUi()` 建立界面，再创建数据库及编辑器、编译运行、题目管理、AI 助教、错因复习、考题管理等对象，并把需共享的对象（如数据库、EditorManager）注入相应控制器；界面通过 `QStackedWidget` 在 coding 与 review 两页间切换。

编码页面由项目文件树（`QFileSystemModel`，过滤常用源文件）、多标签代码编辑区、底部编译日志区与右侧 AI 助教面板组成；助教面板可显隐，其动态 UI 由 `CoachPanelUI` 单独封装为配置页与结果页，以减少主窗口的界面代码。`connect_signals()` 集中把菜单与按钮连接到对应模块：编辑类操作接 `EditorManager`、编译接编译流程、题目录入与查看接 `ProblemController`、助教接 `CoachController`，编译输出则经信号交 `PanelManager` 显示。

2.3 编辑器与文件管理模块

编辑器与文件管理模块由 `CodeEditor`、`EditorManager` 与 `FileManager` 组成。`CodeEditor` 在基础编辑组件上增加 C/C++ 语法高亮与补全，并支持代码行高亮——按助教步骤给出的行号高亮对应代码并移动光标；它将编辑器自身的选择效果与 AI 高亮分别保存再合并显示，避免二者相互覆盖。

`EditorManager` 管理标签页及其对应的 `CodeEditor`：打开文件时先查重以免重复打开，未保存文件在标签标题以星号标记、关闭时提示保存以防误删，并统一封装保存、另存、全部保存、撤销、重做、剪切、复制、粘贴等操作；同时向其他模块提供当前文件路径与代码内容，使它们无需直接访问编辑器控件。`FileManager` 则负责底层文件读写，统一采用 UTF-8 编码以正确保存中文。

2.4 编译运行模块

编译运行模块由 `CppRunner` 与 `PanelManager` 组成。点击编译运行后，主窗口先保存文件、清空旧日志，再交由 `CppRunner`；后者在校验文件已保存且为 `.cpp` 后，用 `QProcess` 以 C++17 标准异步调用 `g++`（开启常用警告），并通过信号读取标准输出/错误送入底部日志，从而在编译期间保持界面响应。编译成功则在独立命令行窗口启动可执行文件，因而能正常支持需从键盘读取数据的程序。`PanelManager` 负责日志显示：`clear_all()` 清空、`append_output()` 追加并滚动到最新。

2.5 题目管理模块

题目管理由 `ProblemController` 负责，建立当前 C++ 文件与题目的绑定并完成录入、修改、分析与查看。绑定以文件路径为键（操作前要求文件已保存且为 `.cpp`），使重新打开同一文件仍能查到题目。录入或修改时按路径查库：已绑定则回显原题面，否则新建记录；确认后先存原文并建立绑定，再调用 AI 做题面结构化（标题、背景、输入输出、样例、提示等）与题目分析（概括、知识点、限制、常见坑、思路、难度）。

由于模型请求较慢，分析放在 `ProblemAnalyzeWorker` 加 `QThread` 后台执行，经进度 / 完成 / 失败信号回传并更新库中的分析状态（`analyzing` / `done` / `failed`）；控制器持有线程与工作对象引用直至任务结束再释放，避免线程被提前销毁。查看题目时按状态分别处理（未绑定则提示录入、分析中则提示稍候、失败则可重试、完成则展示），展示时优先用结构化结果、不完整时回退原文，从而把代码文件、题目原文与分析结果保存在一起，为 AI 助教提供完整题目背景。

2.6 AI 助教模块

AI 助教模块由 `CoachController` 与 `CoachPanelUI` 组成，前者负责收集上下文、创建会话、处理分步提示与保存诊断，后者负责界面与用户输入。启动前，控制器收集文件路径、代码、绑定题目及其分析、最近一次编译信息，以及用户在面板填写的程序输入 / 实际输出 / 期望输出 / OJ 结果 / 补充说明（文件未保存或未绑定题目会先提示）；若最近为编译错误，则优先送编译器错误信息、弱化无关输入输出，使 AI 聚焦编译问题。

系统提供纠错调试、下一步提示与自动判断三种模式。开始会话时先存代码记录并创建诊断，再按模式调用相应 AI 会话，请求经通用工作对象加 `QThread` 执行。助教不一次给出完整答案，而是通过会话逐步返回步骤（含标题、关注点、引导、思考问题、预期发现、建议操作与代码行号）；控制器收到后先落库再渲染，若含行号则调用 `EditorManager.highlight_lines()` 高亮对应代码、切步时

更新或删除。下一步提示模式要求学生先改代码：点击继续时比对新旧代码，未变则提示先修改，已变则更新上下文再取下一条。

纠错调试结束后，控制器保存完整诊断（错误类型、摘要、知识点、疑似位置、建议、置信度等），并据题目、代码与诊断生成错因卡片（记录根因、错误模式、复习问题、提示、规避方法、标签、优先级等），供复习模块继续使用。

2.7 复习与考试模块

复习与考试模块主要由 `ReviewManager`、`ExamManager`、`ExamAnswerDialog`、`ExamListDialog`、`ExamHistoryDialog` 和 `CodeJudge` 等类组成，负责将编码模式中积累的错误转化为可复用的学习资源，并实现自动组卷、答题与判题的完整流程。该模块是 AC-Coach 学习闭环的终点，也是新一轮学习的起点。

2.7.1 错因库与复习材料

`ReviewManager` 是复习模块的核心控制器，负责错因卡片的展示、筛选、状态管理以及复习材料的生成。

在 `review` 模式下，`refresh_mistake_list()` 方法根据用户选择的筛选条件（全部/未掌握/已掌握）、知识点分类和关键词搜索，从数据库中查询对应的错因记录。每条错因记录被封装为一个 `MistakeCard` 控件，该控件继承自 `QWidget`，内部包含错误类型标签、错误描述、知识点标签以及一个“已掌握/未掌握”的切换按钮。当用户点击切换按钮时，`MistakeCard` 会调用 `Database.mark_mistake_mastered()` 更新数据库中的掌握状态，并同步更新按钮的显示文本和样式。此外，`update_knowledge_combo()` 方法遍历所有错因记录，提取其中的 `knowledge_points` 字段（以 JSON 数组形式存储），去重后填充到知识点筛选下拉框中，方便用户按知识点过滤错因。

点击右上角的“生成复习资料”，`generate_review_material()` 方法首先从数据库获取所有错因记录（含已掌握），将其转换为 `error_cards` 列表，然后弹出 `get_review_config_dialog()` 让用户输入额外要求和是否启用深度思考。确认后，`ReviewManager` 创建 `ReviewMaterialThread` 在后台线程中调用 LLM 的 `generate_review_material` 函数。生成完成后，通过 `review_generated` 信号触发 `on_review_generated`，将结果格式化为 HTML 并在自定义对话框中展示。对话框底部提供“生成考题”按钮，可直接跳转到试卷生成流程，实现了复习与练习的无缝衔接。

2.7.2 考题生成与管理

`ExamManager` 负责考题的自动生成、保存和历史管理。

生成好复习资料后，点击“生成考题”，`get_exam_config_dialog()` 会弹出一个配置对话框，用户可设置试卷名称（留空自动使用时间戳）、三类题目（短代码补全、长代码补全、从零重写）的数量以及额外要求。配置完成后，`generate_exam()` 方法会校验题目总数是否大于零，然后创建 `ExamPaperThread` 在后台调用 LLM 的 `generate_exam_paper` 函数。生成完成后，`on_exam_generated()` 方法将 `paper` 中的每道题目通过 `save_questions_to_db()` 保存到 `exam_attempts` 表中。每道题记录包括题目类型、标题、题面（`problem_statement`）、代码模板（`code_template`）、标准答案（`standard_code`）、隐藏测试用例（`hidden_tests`）以及初始状态（`in_progress`）。随后弹出 `ExamListDialog` 展示所有题目，用户可选择其中一题开始作答。

在 review 页面，用户还可点击右上角的“查看已生成的考题”，`show_exam_history()` 方法会从数据库获取所有 `exam_attempts` 记录，按 `exam_id` 分组后构建考试列表，通过 `ExamHistoryDialog` 展示。用户可以选择某次考试并查看其中的所有题目，也可以删除整张试卷及其所有答题记录。

2.7.3 答题与判题

`ExamAnswerDialog` 是一个集题目展示、代码编辑、提交判题于一体的对话框。用户在答题过程中可随时点击“保存草稿”将当前代码保存到数据库，关闭对话框时也会自动保存。用户点击“提交”后，`submit_code()` 方法首先保存草稿，然后获取当前题目的 `standard_code` 和 `hidden_tests`。若两者均存在，则创建 `JudgeThread` 在后台线程中执行判题。`JudgeThread` 内部调用 `CodeJudge.judge()` 进行实际的编译、运行和比对。`CodeJudge` 类实现了与主编辑器环境隔离的沙箱判题机制。其 `judge()` 方法执行以下步骤：

1. **编译用户代码：**使用 `tempfile.NamedTemporaryFile` 在临时目录中创建 `.cpp` 文件，调用 `g++` 编译，若编译失败则返回 `CE`。
2. **编译标准答案代码：**同样方式编译 `standard_code`，若失败则返回 `CE`。
3. **逐用例执行：**对每个 `hidden_tests` 中的输入，使用 `subprocess.run` 分别运行用户程序和标准答案程序，捕获其标准输出。若用户程序超时则返回 `TLE`，若运行时异常则返回 `RE`。
4. **输出比对：**对每个测试用例，比较用户输出与标准输出（`strip()` 后比较），若不一致则返回 `WA` 并记录失败的测试用例索引。
5. **清理：**无论成功与否，最后都会删除临时生成的 `.cpp` 和 `.exe` 文件。

判题完成后，`on_judge_finished()` 根据结果（`AC/WA/RE/TLE/CE`）显示不同的提示信息。若结果为 `AC`，对话框自动关闭；若为 `WA`，则显示用户输出与期望输出的对比，帮助学生定位问题。提交记录会保存到 `exam_submissions` 表中，并将对应 `exam_attempts` 的状态更新为 `completed`。

2.8 LLM 大模型调用层（llm 模块）

LLM 调用层是 AC-Coach “智能”的核心，封装在 `llm/` 文件夹中，负责把题目、代码、运行结果等原始信息组织成合适的提示词，调用 `DeepSeek` 大模型，并把返回内容整理成程序其他部分可以直接使用的结构化数据。它对上层完全屏蔽了“如何与模型对话”的细节：上层只需调用若干高层入口函数或会话对象。整个模块集中在 `llm.py` 一个文件中，涵盖全部与大模型交互的功能：调用原语、上下文构造、题面结构化、题目分析、错误诊断、分步引导、下一步提示、复习与自动出题，以及三个会话类。

2.8.1 调用基础设施

模块底层提供一组与业务无关的通用能力：`get_client()` 每次调用才从 `QSettings` 读取 `deepseek/api_key` 构造客户端，密钥不缓存、不层层传递，既避免硬编码泄露又能即时生效；`call_json()` 发起返回 JSON 的非流式请求、`stream_content()` 做流式逐块产出；`extract_json()` 剥离 Markdown 围栏并在失败时正则兜底、`_call_json_retry()` 在返回非法 JSON 时加严约束自动重试；`thinking_`

`body()` 归一化 DeepSeek 的深度思考开关。上下文由 `build_context()` 统一装配（题面、分析、代码、输入输出、报错、OJ 等），其中 `add_line_numbers()` 为代码加行号以便精确定位、`cut()` 截断超长文本以控 token、`normalize_three()` 把模型名与思考开关归一化为“诊断—复核—引导”三阶段的三元组。

2.8.2 编码模式：诊断—复核—分步引导流水线

面向“纠错调试”的核心是一条由 `prepare()` 组织的三阶段流水线（图 2）。其要点在于两处幻觉机制：`diagnose_error()` 初诊后由 `review_diagnosis()` 做自我复核，校正行号不准、证据不足等问题；且所有提示词均以严格 System Prompt 约束模型只输出可展示内容、不给完整代码、不输出思维链，贴合“引导而非代写”的定位。

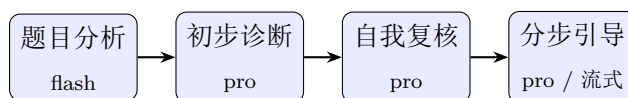


图 2: 纠错调试的四段式生成流水线（标注了各段所用模型）

引导步骤采用流式增量协议：`generate_guide_steps_stream()` 让模型每完成一步即吐出 `<step>...</step>` 程序以正则 `STEP_RE` 实时切分产出。为安全接入 Qt，`DebugGuideSession` 在后台守护线程运行生成器、把每步放入 `queue.Queue`，界面线程用 `next_step(timeout)` 惰性取出并以“点击下一步”节奏展示，末尾以 `END` 哨兵结束——这一生产者—消费者结构兼顾界面流畅与分步交互。

2.8.3 编码模式：下一步提示与自动路由

面向“尚未写完”的学生，`NextHintSession` 只在 `generate_next_hint_step()` 中给出一个很小的下一步提示，并将 `waiting_for_update` 置真——此后 `next_step()` 会返回 `None`，强制学生先真正修改代码，再通过 `update_context()` 提交新代码、重新路由后才继续下一条提示，从机制上杜绝了“连点下一步套出完整题解”。在两种模式之上，`AutoCoachSession` 先用 `analyze_student_need()` 作为路由判断器，依据代码完成度与报错情况自动决定进入 `debug` 还是 `next_hint`，再委托给相应会话，对应界面中的“自动判断”模式。三个会话类由 `start_*_session()` 工厂函数统一创建。

2.8.4 复习模式：错因聚类与自动出题

复习与出题能力形成“压缩—聚类—规划—生成”链路：`summarize_error_record()` 把单次记录压成错因卡片，`analyze_review_history()` 按知识点聚类作为共同基础，`generate_review_material()` 产出分知识点复习材料，`plan_exam_paper()` 先规划整卷题目规格、再由 `generate_exam_question()` 逐题生成。其中两处设计值得强调：每题返回 `standard_code` 与仅含输入的 `hidden_tests`（`user_view` 不泄露二者、补全题用 `__BLANK_n__` 挖空），使判题可自动编译比对；`generate_exam_paper()` 用 `ThreadPoolExecutor` 并行生成、并将单题失败收进 `failed_questions` 而不中断整卷。

综观全模块，其关键设计可概括为三点：一是严格 JSON 契约加校验闭环，题面结构化更以 `validate_structured_problem()` 逐字段校验是否为原文原样片段、不过则反馈重试（至多三次），抑制改写与臆造；二是模型分层，轻任务用 `deepseek-v4-flash`、重任务用 `deepseek-v4-pro` 以平衡成本与质量；三是教学导向约束，全程“引导而非代答”，使系统区别于普通 AI 问答工具。

2.9 数据持久层

数据持久层由 `Database` 类实现，负责所有数据的结构化存储与查询。该类使用 SQLite 作为底层数据库引擎，封装了建表、迁移和 CRUD 操作，是 MVC 架构中的 Model 层核心。

2.9.1 数据库初始化与迁移

`Database` 类在实例化时接受可选的 `db_path` 参数（默认使用项目根目录下的 `accoach.db`），并调用 `_init_database()` 完成初始化，依次创建以下核心表：

表 3: 数据库核心表及其用途

表名	用途
<code>problems</code>	存储题目原文、结构化题面（标题/背景/描述/输入输出描述/样例等）、题目分析结果（摘要/知识点/限制条件/常见坑点/建议思路/难度）以及分析状态
<code>problem_files</code>	建立 <code>.cpp</code> 文件路径与 <code>problems</code> 表之间的绑定关系，支持 <code>ON CONFLICT UPDATE</code> 实现文件路径的唯一约束
<code>code_records</code>	存储每次助教会话的代码快照、语言、输入输出上下文（ <code>program_input/program_output/expected_output</code> ）、错误信息、OJ 结果和运行状态
<code>diagnoses</code>	存储每次 AI 诊断的完整记录，包括错误类型、错误摘要、知识点、疑似位置、置信度、调试建议和会话模式
<code>guide_steps</code>	存储诊断过程中的每一步引导内容，包括步骤编号、标题、引导文本、行号范围、引导问题、预期发现和下一步操作建议
<code>mistake_library</code>	存储错因卡片，包括错误类型、错误描述、错误代码片段、知识点、掌握状态、复习优先级和错因卡片各字段
<code>exam_attempts</code>	存储试卷题目及用户答题状态，包括题目类型、题面、代码模板、用户代码、标准答案、隐藏测试用例和完成状态
<code>exam_submissions</code>	存储每次提交的代码和判题结果，用于记录用户的答题历史

2.9.2 核心数据操作

题目相关：`add_problem()` / `update_problem()` 整体写入或更新题目（含结构化字段与分析结果），`get_problem()` / `get_all_problems()` 查询单条或全部题目，`bind_problem_file()` 以 `INSERT ... ON CONFLICT UPDATE` 等地绑定文件与题目、`get_problem_by_file()` 反向查询，`set_problem_analysis_status()` 维护分析状态（`analyzing/done/failed`）以驱动 Check 流程。

代码记录相关：`add_code_record()` 插入代码快照，`update_code_context()` 供 `next_hint` 模式在学生改码后更新上下文，`ensure_code_record()` 按题目与代码内容复用最近记录以避免重复插入，`get_latest_code_record_by_problem()` 取最近记录以延续上下文。

诊断与步骤相关：`add_diagnosis()` 创建诊断（初始 `starting`），`set_diagnosis_mode_status()` 维护会话模式（`debug/next_hint`）与运行状态，`update_diagnosis()` 在结束后写入完整诊断结果，`add_guide_step()` / `get_guide_steps()` 保存与按序读取每一步引导。

错因库相关: `add_mistake()` 插入错因卡片 (含根因、错误模式、复习问题、优先级等字段), `get_all_mistakes()` 按掌握状态过滤查询, `mark_mistake_mastered()` 供 `MistakeCard` 更新掌握状态。

考试相关: `save_exam_attempt()` 幂等地插入 / 更新答题记录, `get_exam_attempt()` / `get_all_exam_attempts()` 查询, `add_exam_submission()` 记录提交历史并将对应记录置为 `completed`, `delete_exam_by_id()` 删除整卷及其所有答题记录。

3 小组成员分工情况

表 4: 小组成员分工

姓名	学号	主要负责工作
蔡东辰	2400011001	负责 review 模式的 UI 和功能以及数据库的开发
田特	2500011169	coding 模式的 UI 与基本功能设计开发
刘家宏	2500010968	项目及功能设计与大模型部分

4 项目总结与反思

4.1 项目总结

AC-Coach 实现了一个围绕“程序设计实习”课程的完整学习闭环工具。在编码模式下, 它集成了 C++ 文件的编辑、g++ 编译运行与编译错误解析, 可将题目原文录入并由大模型自动结构化与分析, 再由 AI 助教以“自动判断 / 纠错调试 / 下一步提示”三种模式给出分步引导; 在复习模式下, 它把每次调试沉淀为错因卡片, 并据此生成复习材料与可自动判题的程序填空 / 重写题, 从而把“读题—编码—运行—调试—复盘—复习”串成一条闭环。

与普通 AI 问答工具相比, 本项目的特色在于**教学导向**而非直接代答: 助教通过代码行高亮与分步提示引导学生自己发现错误, 下一步提示模式更要求学生先修改代码才能继续。在工程实现上, 项目采用以 `MainWindow` 为组合根、各 `Manager` / `Controller` 经信号协作的清晰分层结构, 所有大模型调用统一封装在 `llm` 模块并置于后台线程执行, 配合严格的 JSON 契约、题面原样校验与模型分层等手段, 在保证界面流畅的同时提升了 AI 输出的稳定性与可用性。

4.2 不足与改进方向

受时间与精力所限, 当前版本仍存在一些不足, 也留下了后续可改进的空间:

- **语言支持与判题安全:** 目前仅支持 C++; `CodeJudge` 直接编译并运行用户代码, 缺少时间 / 内存限制与沙箱隔离, 面对死循环或恶意代码不够安全。后续可引入资源限制与沙箱机制, 并扩展对更多语言的支持。
- **AI 输出的稳定性与响应时延:** 系统的智能能力依赖 DeepSeek 在线大模型, 由此带来两类偶发问题。其一是输出不稳定: 模型可能返回不完全符合约定 JSON 结构的内容, 或给出不准确的可疑行号乃至臆造信息——尽管已通过容错解析、失败重试、题面原样校验与诊断自我复核等手段尽

量抑制，仍无法完全杜绝，极端情况下多次重试后仍可能失败。其二是响应时间偏长：一次“召唤助教”或整卷出题往往需要“诊断—复核—引导”“分析—规划—逐题生成”等多次串行的重模型调用，在网络较慢或题目较复杂时耗时明显（尽管界面已通过后台线程保持响应）。后续可对题目分析等结果做缓存、进一步流式反馈中间进度、精简或并行化调用链，并在必要时降级到更快的模型来缓解。

- **提示词硬编码，扩展性受限：**目前各环节的系统 / 用户提示词均以字符串字面量的形式内嵌在 `llm.py` 中，而非从外部数据库或配置文件加载。这使得提示词工程与代码强耦合：调整措辞、新增题型，或将系统适配到其他课程 / 编程语言，都需要改动源码并重新发布，也不便对提示词做版本管理与对比测试。后续可将提示词外置为数据库记录或配置模板（配合变量占位符），在运行时按课程动态加载，从而支持热更新与更灵活的扩展。